Progress Report: Multi-Task ALIGNN Representation Learning

ALIGNN Reimplementation Project

May 2026

1 Purpose

This report documents the recent shift from per-target ALIGNN training to a shared-encoder multi-task ALIGNN experiment. The goal was to test whether supervised representation learning across thermodynamic, electronic, and mechanical targets improves data efficiency and downstream prediction quality compared with separately trained single-task models.

The central hypothesis was:

> A shared ALIGNN graph representation trained across multiple JARVIS targets learns transferable crystal features that improve low-data and target-specific performance.

2 Targets

The multi-task experiment used five JARVIS `dft_3d` targets:

Target	Property type
`formation_energy_peratom`	Thermodynamic stability energy
`ehull`	Thermodynamic metastability measure
`optb88vdw_bandgap`	Electronic band gap
`mbj_bandgap`	Electronic band gap
`bulk_modulus_kv`	Mechanical elastic property

3 Model Change

The original training path used separate single-task ALIGNN models. Even when the architecture was identical, each target had its own independently trained weights:

```
formation_energy_peratom -> ALIGNN encoder A -> one scalar
ehull                    -> ALIGNN encoder B -> one scalar
mbj_bandgap              -> ALIGNN encoder C -> one scalar
bulk_modulus_kv          -> ALIGNN encoder D -> one scalar
```

The new multi-task model uses one shared `ALIGNNGraphEncoder` and one prediction head per target:

```
                          -> formation_energy_peratom head
                         -> ehull head
graph + line graph -> shared ALIGNNGraphEncoder -> optb88vdw_bandgap head
                         -> mbj_bandgap head
                          -> bulk_modulus_kv head
```

In this codebase, the shared encoder means the full `ALIGNNGraphEncoder` in `src/alignn/models/alignn_mo` It includes:

- `CrystalFeatureEncoder`, which embeds atom, bond-distance, and angle features.
- A stack of ALIGNN atom-graph and line-graph update layers.
- A stack of edge-gated graph convolution layers.
- Graph pooling to produce one crystal-level feature vector.

Thus the shared part is larger than only the atom or bond encoder. It is the complete structure-representation block up to the pooled graph feature vector. Each target-specific head maps that shared graph feature vector to one scalar property.

4 Implemented Work

The following implementation work was completed.

4.1 Shared-Encoder Model

Added `ALIGNNGraphEncoder` and `MultiTaskALIGNNModel` in `src/alignn/models/alignn_model.py`. The single-task `ALIGNNModel` was kept compatible by making it use the same `ALIGNNGraphEncoder` followed by a single linear readout.

4.2 Multi-Task Data Path

Added target-labeled graph samples and a multi-task collate path in `src/alignn/data/dataset.py`. The first implementation uses homogeneous target batches: each batch contains samples for one target, while the training loop alternates across target loaders.

4.3 Training Logic

Added `train_multitask_alignn` in `src/alignn/train/trainer.py`. The trainer now supports:

- Per-target standardization for multi-task losses.
- Homogeneous per-target data loaders.
- Target-specific heads with a shared encoder.
- Multi-task checkpoint saving.
- Encoder checkpoint saving.
- Single-task fine-tuning from a multi-task checkpoint.
- Low-data train fractions for single-task and multi-task comparisons.
- Test metrics export for analysis.

4.4 CLI and Scripts

Added CLI commands and experiment scripts:

- `alignn-train-multitask`
- `alignn-overfit-multitask`
- `--pretrained-multitask-checkpoint`
- `--train-fraction`
- `scripts/run_multitask_phase0_baselines.sh`
- `scripts/run_multitask_phase4_joint.sh`
- `scripts/run_multitask_phase5_finetune.sh`
- `scripts/run_multitask_phase6_low_data_matrix.sh`
- `scripts/run_multitask_phase7_ablations.sh`
- `scripts/analyze_multitask_results.py`

After the long run completed, a worker-control improvement was also added for future runs:

- `--num-workers` for single-task training.
- `--num-workers` for multi-task training.
- CUDA data loaders now use pinned memory when appropriate.

This was verified locally with `uv run python -m compileall src` and CLI help checks for both training commands.

5 Validation Progress

Several smoke tests passed before launching the full matrix:

- Source compiled successfully on the remote instance.
- `alignn-train-multitask --help` worked through module invocation.
- `alignn-overfit-multitask --help` worked.
- A tiny multi-task smoke run completed and wrote checkpoint, encoder checkpoint, history, predictions, and metrics.
- A tiny multi-task overfit run completed.
- A tiny single-task fine-tune run loaded encoder weights from a multi-task checkpoint and completed.
- The analysis script aggregated metrics into `results/tables/multitask_metrics_summary.csv`.

The remote console entrypoint became stale after a restart, so long jobs were launched with:

```
uv run python -m alignn.cli ...
```

instead of the stale console-script path.

6 Completed Experiment Matrix

The completed Phase 6 low-data run used:

- Seeds: `123`, `234`, `345`
- Train fractions: `0.10`, `0.25`, `0.50`, `1.0`
- Targets: five targets listed above
- Epochs: `10`
- Batch size: `16`
- Hidden dimension: `64`
- ALIGNN layers: `4`
- GCN layers: `4`
- Loss: `smoothl1`
- Scheduler: `onecycle`

The final aggregation contained `195` Phase 6 metric rows:

Run type	Metric rows
Scratch single-task	60
Fine-tuned single-task	60
Joint multi-task	60
Full-data pretrain joint	15

7 Meaning of Compared Methods

7.1 Scratch

`scratch` means a normal single-task ALIGNN model was initialized randomly and trained only on one target.

7.2 Joint

`joint` means one `MultiTaskALIGNNModel` was trained directly across all five targets. The shared `ALIGNNGraphEncoder` received gradients from every target, while each target had its own prediction head.

7.3 Fine-Tune

`finetune` means two-stage training:

1. Train a full-data multi-task model across all five targets.
2. Load that shared encoder into a normal single-task ALIGNN model.
3. Continue training on only one target and one train fraction.

Fine-tuning therefore tests whether the multi-task encoder learned a transferable initialization for a target-specific model.

8 Phase 6 Results

The table below reports mean test MAE across the three seeds. Lower is better.

Fraction	Target	Scratch	Fine-tune	Joint
0.10	bulk_modulus_kv	25.35	**19.83**	23.97
0.10	ehull	0.1050	0.1018	**0.0963**
0.10	formation_energy_peratom	0.3371	**0.1980**	0.3176
0.10	mbj_bandgap	1.287	**0.934**	1.201
0.10	optb88vdw_bandgap	0.4261	0.3930	**0.3313**
0.25	bulk_modulus_kv	21.64	**18.33**	20.49
0.25	ehull	0.0833	0.0823	**0.0798**
0.25	formation_energy_peratom	0.2651	**0.1527**	0.2453
0.25	mbj_bandgap	1.162	**0.830**	1.119
0.25	optb88vdw_bandgap	0.3454	0.3128	**0.2862**
0.50	bulk_modulus_kv	18.88	**16.83**	17.82
0.50	ehull	0.0711	0.0737	**0.0691**
0.50	formation_energy_peratom	0.1957	**0.1316**	0.1761
0.50	mbj_bandgap	1.054	**0.799**	0.992
0.50	optb88vdw_bandgap	0.3188	0.3033	**0.2759**
1.00	bulk_modulus_kv	16.83	**16.02**	16.65
1.00	ehull	0.0692	0.0705	**0.0587**
1.00	formation_energy_peratom	0.1353	**0.1138**	0.1389
1.00	mbj_bandgap	0.975	**0.756**	0.862
1.00	optb88vdw_bandgap	0.2900	0.2764	**0.2618**

9 Observed Improvements

9.1 Fine-Tuning Improvements

Fine-tuning from the multi-task encoder was the strongest overall strategy. Compared with scratch single-task training, the average percent MAE changes across fractions and seeds were:

Target	Mean MAE improvement	Win rate vs scratch
formation_energy_peratom	33.0%	100%
mbj_bandgap	25.7%	100%
bulk_modulus_kv	13.2%	100%
optb88vdw_bandgap	6.7%	100%
ehull	-0.4%	50%

The largest low-data gains were:

- formation_energy_peratom at 10% data: MAE improved from 0.3371 to 0.1980, about 41% better.
- mbj_bandgap at 10% data: MAE improved from 1.287 to 0.934, about 27% better.
- bulk_modulus_kv at 10% data: MAE improved from 25.35 to 19.83, about 22% better.

9.2 Joint Multi-Task Improvements

Joint multi-task training also improved over scratch in most cells, but its gains were smaller and more target-dependent. Average improvements over scratch were:

Target	Mean MAE improvement	Win rate vs scratch
optb88vdw_bandgap	15.6%	100%
ehull	7.1%	75%
mbj_bandgap	6.7%	92%
formation_energy_peratom	5.1%	83%
bulk_modulus_kv	4.3%	67%

10 Interpretation

The results support the main hypothesis: multi-task supervised representation learning produced useful transferable features. Scratch training was not the best mean method for any target/fraction cell.

The best strategy depends on the target:

- Use multi-task pretraining plus single-task fine-tuning for `formation_energy_peratom`, `mbj_bandgap`, and `bulk_modulus_kv`.
- Use direct joint multi-task prediction for `ehull` and `optb88vdw_bandgap`.

This suggests that some properties benefit most from a shared representation followed by target-specific specialization, while others benefit from remaining inside the jointly trained model.

11 Caveats

The current conclusions should be treated as directional rather than final production-quality results:

- All Phase 6 runs used a short `10`-epoch budget.
- Fine-tuning used a full-data multi-task pretrain checkpoint, so it had representation exposure beyond the target-specific low-data subset.
- The experiment measured held-out MAE and RMSE, but further per-sample error analysis would help identify transfer and negative-transfer regimes.
- The remote console script became stale after restart; module invocation was used to avoid that environment issue.

12 Next Steps

Recommended next steps are:

1. Run longer budgets for the best method per target.
2. Compare fine-tuning against stronger tuned single-task baselines, not only short-budget scratch runs.
3. Add target-group ablations, especially thermodynamic-only and electronic-only groups.
4. Analyze per-sample wins and losses to identify which crystal families benefit from transfer.
5. Preserve the `--num-workers` setting in future remote launches to improve data-loading throughput.

13 Artifact Locations

Important artifacts are:

- Plan: `MULTITASK_ALIGNN_PLAN.md`
- Aggregated metrics: `results/tables/multitask_metrics_summary.csv`
- Target correlations: `results/tables/multitask_target_correlations.csv`
- Phase 6 log: `results/logs/rest_phase6_allseeds_pipeline.log`
- Post-run analysis log: `results/logs/rest_phase6_post_analysis.log`
- Model code: `src/alignn/models/alignn_model.py`
- Training code: `src/alignn/train/trainer.py`
- CLI code: `src/alignn/cli.py`